UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

ACTIVIDAD 2. PROBLEMAS Y ESPACIOS DE BÚSQUEDA

Alumno: Eduardo Parrilla Toledano

Grupo: 172

Docente: Mtro. Javier García Lara

1. Introducción

Un problema de búsqueda consiste en encontrar una secuencia de acciones desde un estado inicial hasta un estado objetivo. En un grafo, las localidades son nodos y las conexiones son

aristas con costos. Dijkstra encuentra una ruta de costo mínimo cuando los costos no son negativos.

2. Planteamiento del problema

- Inicio: Atlautla.
- Destino: Amecameca. La ruta termina aquí; no continúa a Ciudad de México.
- Acciones: desplazarse por una conexión disponible.
- Costo: valor ficticio para la práctica; no representa kilómetros reales.
- Objetivo: encontrar la ruta de menor costo entre ambas localidades.

3. Grafo de ejemplo

La conexión Atlautla ↔ Amecameca tiene un costo didáctico de 8 unidades. Este número no es una distancia vial real.

Origen	Destino	Costo de ejemplo
Atlautla	Amecameca	8

4. Código Python documentado

```
# ACTIVIDAD 2: RUTA CON EL ALGORITMO DE DIJKSTRA
import heapq # Permite extraer primero el nodo con menor costo.

def dijkstra(grafo, inicio, objetivo):
    """
    Encuentra la ruta de menor costo.
    grafo: diccionario de nodos y vecinos (vecino, costo).
    inicio: localidad de salida.
    objetivo: localidad de llegada.
    Devuelve la ruta y su costo total.
    """
    cola = [(0, inicio)]       # Cola: (costo acumulado, nodo).
    distancias = {inicio: 0}   # Menor costo conocido desde el inicio.
    anteriores = {}            # Permite reconstruir la ruta final.
    visitados = set()          # Nodos ya procesados.

    while cola:                # Repetir mientras haya nodos pendientes.
        costo_actual, nodo = heapq.heappop(cola)

        if nodo in visitados:   # Ignorar entradas repetidas.
            continue
        visitados.add(nodo)     # Marcar el nodo como procesado.

        if nodo == objetivo:    # Si llegamos al destino, terminar.
            break

        # Revisar cada conexión que sale del nodo actual.
        for vecino, costo_tramo in grafo.get(nodo, []):
            if costo_tramo < 0:
                raise ValueError("Dijkstra no admite costos negativos")

            nuevo_costo = costo_actual + costo_tramo

            # Actualizar solo si encontramos una ruta más barata.
            if nuevo_costo < distancias.get(vecino, float("inf")):
                distancias[vecino] = nuevo_costo
                anteriores[vecino] = nodo
                heapq.heappush(cola, (nuevo_costo, vecino))

    # Si el destino no tiene distancia registrada, no hay ruta.
    if objetivo not in distancias:
```

```
        return None, float("inf")

    # Reconstruir la ruta desde el destino hasta el inicio.
    ruta = []
    actual = objetivo
    while actual != inicio:
        ruta.append(actual)
        actual = anteriores[actual]
    ruta.append(inicio)
    ruta.reverse()            # Poner los nodos en orden de viaje.

    return ruta, distancias[objetivo]


# Grafo de ejemplo: solo se requiere llegar hasta Amecameca.
# El costo 8 es ficticio y se usa para demostrar el algoritmo.
grafo_transporte = {
    "Atlautla": [("Amecameca", 8)],
    "Amecameca": [("Atlautla", 8)]
}

inicio = "Atlautla"           # Punto donde comienza el recorrido.
objetivo = "Amecameca"        # Punto donde termina el recorrido.
ruta, costo_total = dijkstra(grafo_transporte, inicio, objetivo)

# Mostrar el resultado en la terminal.
print("Ruta de transporte")
print("Inicio:", inicio)
print("Destino:", objetivo)
if ruta is None:
    print("No se encontro una ruta.")
else:
    print("Camino encontrado:", " -> ".join(ruta))
    print("Costo total de ejemplo:", costo_total)
```

5. ¿Qué hace el programa?

1. Representa las localidades y conexiones mediante un diccionario llamado grafo_transporte.
2. Inicia la búsqueda en Atlautla con costo cero.
3. Examina la conexión hacia Amecameca y calcula 0 + 8 = 8.
4. Al llegar al destino, termina la búsqueda.
5. Reconstruye e imprime la ruta final y su costo.

6. Resultado esperado

```
Ruta de transporte
Inicio: Atlautla
Destino: Amecameca
Camino encontrado: Atlautla -> Amecameca
Costo total de ejemplo: 8
```

7. Complejidad

Con una lista de adyacencia y una cola de prioridad, la complejidad temporal aproximada es O((V + E) log V), donde V representa los nodos y E las conexiones. El ejemplo es pequeño, por lo que el cálculo es inmediato.

8. Conclusión

Dijkstra permite encontrar una ruta de costo mínimo cuando los costos son no negativos. En esta actividad, el recorrido va únicamente de Atlautla a Amecameca. El costo 8 es ficticio y solo sirve para demostrar el funcionamiento.